

CANDY DIALOGUE ENGINE

Dispositions

1.0.0

In a game, dispositions usually represent the relationship between an NPC and the player, such as Neutral, Like, or Dislike.

Disposition usually influences dialogue: an NPC will not speak or act the same way towards the player depending if they are friends or enemy. Therefore, dialogue becomes conditional on dispositions.

Candy DE provides condition commands that emulate 'if', 'elif' or 'else' in GDScript, but setting up such condition chains each time a dialogue line is dependent on NPC disposition would add unreasonable work and complicate dialogue scripts with a lot of branching.

Instead, Spoken Lines have a built-in convenient method for handling dispositions: the Dispositions field.



Basic Method

To use the Dispositions field, just write a disposition inside.

Then, in the **actors** dictionary (in [Candy_Database.gd](#)), update the relevant actor's disposition at runtime.

When the engine reaches the line, if the speaker's disposition matches the line's, the line will be displayed. If the dispositions don't match, the line will be skipped.

You can not only make actors speak or skip lines this way, but you can also make them choose one line to speak among multiple, according to their current disposition:

Like	:	Alice	:	Hey! Nice to see you!
Dislike	:	Alice	:	Go away.

For example, in the image above, if Alice has the 'Like' disposition assigned, she'll say "Hey! Nice to see you!". If she has the 'Dislike' disposition, she will say "Go away." instead.

Advanced Options

Dispositions can go a lot further. Note that the following is an advanced feature, which you don't need to learn unless you want to use it.

For one thing, you can assign multiple dispositions to a line or to an actor: just separate them by a comma (,) inside strings, like so:

"Like, Friendly, Love"

Second, you can use prefixes to tell the engine how to process dispositions:

- No prefix** → At least one disposition must be a match.
- =** → The line's and the speaker's dispositions must be an exact match.
- +** → All of the line's dispositions must be matched by the speaker, but the speaker can have extra dispositions not present in the line.
- → All dispositions assigned to the speaker must be in the line. The speaker is not required to have any dispositions at all, or all of the line's dispositions.
- (n)** → Exactly n dispositions must match. No more, no less.
- =(n)** → Exactly n dispositions must match, and the speaker can't have any extra dispositions not listed.
- +(n)** → n or more dispositions must match.
- (n)** → Less than n dispositions must match.
- !** → None of the dispositions must match.
- !(n)** → Exactly n dispositions must NOT match.
- !=!(n)** → Exactly n dispositions must NOT match, and the speaker can't have any extra dispositions not listed.
- +!(n)** → n dispositions or more must NOT match.
- !(n)** → Less than n dispositions must NOT match.

Those prefixes are placed at the start of the Disposition field and apply to the entire string (i.e. to all dispositions), they can't be assigned to individual dispositions. Like this:

! Like, Friend, Love
=(2) Like, Friend, Love
+!(2) Like, Friend, Love

This may seem like more power than you'll need, and it probably is for most projects. But this is consistent with Candy DE's philosophy: its purpose is to provide tools if you need them, not dictate what you can or cannot do with your dialogues.

Don't let this trick you into thinking dispositions are complicated: just use the basic form (one disposition per line and speaker, no prefixes), and only turn to the advanced options when you need them.

Candy Dialogue Creator

To avoid typing dispositions each time, you can point the Candy Dialogue Creator to an array that contains a list of the dispositions your project uses most often (or you can manually enter dispositions in Candy Dialogue Creator directly).

You can then click the (:) button next to the Dispositions field of a spoken line to display a list of dispositions, and select them just by clicking.

The Candy Dialogue Creator will automatically add a comma and a space (', ') after each disposition it inserts. You can safely ignore this, it won't break anything even if you leave the comma and space after the last disposition: Candy DE will just ignore redundant commas and spaces, and you don't need to delete them manually every time.

By default, the Candy Dialogue Creator will read dispositions from the **creator_disposition_list** in **Candy_Database.gd**.

Variable Substitution

Every part of the Disposition string can be substituted for a variable. For example, you can write:

`%globals.var_1%globals.var_2, globals.var_3`

Where var_1 is one or more operators, and var_2 and var_3 are dispositions.

Further Uses

While this feature is intended for NPC dispositions in the traditional sense, you can find creative uses for it: the values you use as dispositions don't have to represent NPC opinions of the player, they can represent things an NPC wants to do, hobbies, or moods:

`"Like_Player, Starving, Love_Pasta"`

In the example above, only Like_Player is a proper disposition in the usual sense. 'Starving' is a mood or status, and 'Love_Pasta' is a personal taste or an activity the NPC enjoys.

You could use this to make an NPC accept an invitation to go to an Italian restaurant if any of these dispositions are a match: even if the NPC dislikes the player, they'll accept the invitation if they're starving or if they really love the food.

To avoid filling an actor's disposition list with plenty of entries, you can also store actor opinions, moods and tastes in separate variables. Then, before a line like the previous example occurs, use commands to gather potentially relevant data into the actor's list of dispositions. After the spoken

line has been processed, use control commands again to clear the actor's list of dispositions or reset it to what it should ordinarily be.